Systematic Mutation and Black-Box Probing for SQL Injection WAF Bypass: A Live Testbed and Family-Level Analysis

Co Huy Dung, Tran Xuan Quyen
University
School of Information and Communication Technology/Hanoi University of Science and Technology
Hanoi, Vietnam
Dung.CH252592@sis.hust.edu.vn, Quyen.TX252593@sis.hust.edu.vn

***Abstract*—Attackers routinely evade Web Application Firewalls (WAFs) by mutating SQL injection (SQLi) payloads while preserving intent—reinforcement-learning agents such as SSQLi report bypass rates up to 97.39% in controlled settings [1], and discrepancy-based HTTP mutations can bypass edge inspection even when malicious substrings are not obvious in benign-looking framing [2]. Aggregate bypass figures, however, are fragile: they conflate heterogeneous operators, depend on outcome semantics, and can be driven by search bias (which variants are generated and in what order). Defenders and auditors therefore need *reproducible, family-resolved* attack-side measurements tied to explicit labels and open logs.**

This work adopts an offensive (red-team) perspective and reports a toolchain that (i) composes dialect seeds (PostgreSQL- and Oracle-oriented), deployment-specific augmentations, and learned transform rules into a large discrete variant set; (ii) issues black-box HTTP probes against a live AWS WAF–fronted application with per-attempt JSONL logging; and (iii) aggregates outcomes globally and by mutation *family*, enabling comparison of high-volume cohorts against rare but informative tails. On the archived lab log (9,435 attempts), the probe's operational classifier labels 3,721 attempts (39.4%) as bypasses (WAF-pass plus application-visible signals under the script's definition). Family-level heterogeneity is extreme: identity-, case-mix-, and null-byte-oriented "spawn" cohorts reach per-family bypass rates near 89–90% in the export, while an encoding-heavy spawn cohort is largely blocked (≈2.5% bypass), indicating that policy stress concentrates in a subset of syntactic strategies rather than spreading uniformly across mutations.

As a secondary control, we apply repeated RFC percent-decoding to a fixpoint before scoring strings with a small random-forest surrogate; on a 20-request pilot, accuracy stays at 42.9% for both raw and canonicalized inputs, suggesting that naive decoding neither closes the live bypass gap nor rescues a tiny offline model—consistent with attacks exploiting features beyond a single decoding pass. We emphasize that all rates are *conditional* on the experimental channel (URL-visible injection), policy snapshot, and labeling code; the value of the study lies in the explicit methodology, the decomposed statistics, and the artifact that allows independent replication.

***Index Terms*—SQL injection, web application firewall, black-box attack, adversarial mutation, bypass measurement, variant families**

I. Introduction

Web applications remain the primary interface for commerce, identity, and data access; Web Application Firewalls (WAFs) are widely deployed to block common injection classes using signatures and, increasingly, machine-learned components [9], [10]. From an attacker's standpoint, the decisive question is whether a *specific* deployment, with its concrete rules, parsers, and logging, admits a practical path from crafted HTTP bytes to a security-relevant application outcome. Signatures can be evaded by semantically equivalent rewrites; edge and application parsers can disagree on decoding, normalization, and parameter binding [11]–[13]. Because both phenomena are well documented in the literature, the remaining scientific challenge is often *measurement*: under what conditions, and for which mutation *shapes*, does bypass become likely?

This paper contributes an *attack-centric* evaluation that is deliberately operational. We systematically mutate SQLi fragments, probe a production-style AWS WAF deployment in a black-box manner, and analyze outcomes at aggregate and variant-family granularity. The implementation is released as a reproducible artifact (`paper/experiment`), including generators, probe drivers, JSONL traces, and export scripts used to build the tables in Section IV. We do not propose a new defensive product; we instead supply a methodology and empirical distributions that teams can use for red-team prioritization, blue-team regression testing, and benchmarking of future mitigations.

A. Problem Statement

SQLi evasion combines at least two mechanisms: (i) *representation-level* mutation that preserves attacker intent while altering substrings, token boundaries, or encodings seen by a pattern matcher; and (ii) *interpretation-level* divergence, where the same octet sequence is decoded or framed differently by the WAF path and the application [2].

1) Mutation-Centric Evasion: Prior work treats WAFs as targets for semantic-preserving search. SSQLi frames black-box SQLi as reinforcement learning and reports high bypass rates with SAC [1]. GSQLi uses GAN-based generation [7]; WAF-A-MoLE demonstrates mutational fuzzing against ML WAFs [6]. These systems imply a common lesson: success is rarely uniform across operators—some edits collapse under

normalization while others survive. Yet published headline rates seldom decompose by operator family on a live managed WAF with realistic logging.

2) Discrepancy-Aware Attack Surfaces: WAFFLED shows that mutating ostensibly benign HTTP structure can induce large sets of parsing-discrepancy bypasses across major WAFs [2]. Our live probe narrows scope to URL-visible injection (path or query) with deterministic seed remapping. That choice *reduces* one class of discrepancy (parameter-name mismatch) so that residual outcomes more directly reflect how the WAF treats alternate encodings and spellings of the same logical injection slot—a controlled setting for studying representation-level evasion against a fixed policy.

B. Research Questions

We organize the empirical work around three questions:

- **RQ1 (Scale and semantics).** Under an explicit, script-defined notion of bypass, what fraction of probed attempts succeed against a live AWS WAF lab configuration, and how does that fraction relate to unique payload cardinality?
- **RQ2 (Structure).** How much of the bypass mass is concentrated in a small set of mutation families, and which families are systematically suppressed (e.g., encoding-heavy spawns)?
- **RQ3 (Normalization control).** Does a minimal RFC-oriented percent-decoding fixpoint materially change a tiny offline surrogate's ability to separate attack-like from benign-like strings, suggesting trivial decoder alignment would erase the attack surface?

C. Objectives and Contributions

1) **Methodology.** We formalize a black-box attack pipeline—staged variant generation (dialect seeds, AWS-oriented augmentations, learned transforms, optional second-layer transforms), HTTP probing with JSONL logging, and export-driven aggregation—for managed WAF assessment.
2) **Empirical characterization.** On the archived AWS WAF lab aggregate (9,435 attempts), we report an overall bypass rate of 39.4% under the probe's labels, unique payload counts, and family-level bypass rates that reveal sharp heterogeneity between spawn strategies.
3) **Secondary control.** We include a 20-request pilot comparing a random-forest surrogate on raw versus RFC-style canonicalized strings, observing identical 42.9% test accuracy, which cautions against equating single-pass decoding with robust defense.

D. Paper Organization

Section II situates mutation-based SQLi, discrepancy attacks, and defensive context. Section III defines metrics, outcome semantics, and toolchain stages. Section IV answers RQ1–RQ3 with corpus statistics, aggregate and family tables, and the surrogate pilot. Section V discusses validity, defender implications, ethics, and future work.

Layers: lexical SQLi mutations; URL/query encodings; edge WAF normalization; application parser and ORM; observable HTTP/app signals

Fig. 1. Conceptual layering for SQLi WAF bypass experiments (simplified). Each layer can introduce transformations that affect whether a static rule or model "sees" attack tokens.

II. Background and Related Work

Offensive SQLi evaluation sits at the intersection of adversarial search against detectors, HTTP semantics, and operational security measurement. We review strands that motivate *family-resolved* reporting rather than headline bypass percentages alone.

A. Mutation and Learning-Based SQLi Attacks

Adversarial methods search for inputs that preserve attacker intent while evading a detector's decision boundary. SSQLi reports bypass rates up to 97.39% in its experimental setting using SAC [1]. GSQLi uses GAN-based mutation [7]. WAF-A-MoLE applies semantics-preserving fuzzing to stress ML WAFs [6]. These lines of work share a structural insight: the attack space is combinatorial—comment insertion, case folding, encoding nests, whitespace and delimiter tricks, and dialect-specific spellings can be composed. Compositions interact non-monotonically with regex-based and tokenized defenses: an encoding that defeats one rule may trigger another, or may change how a downstream parser tokenizes a string [14].

From a measurement standpoint, the relevant output is therefore not only "bypass rate" but *which* compositions survive, because survivors inform both attacker budgets (which mutations to prioritize) and defender budgets (which rules or tests to add first).

B. Parsing Discrepancies and HTTP-Level Evasion

When components disagree on decoding order, charset, or structural boundaries, attackers can sometimes satisfy benign predicates at the edge while delivering malicious semantics to the application [2]. WAFFLED's large-scale study underscores that HTTP is not a single abstract datatype but a family of implementations. Our experiments deliberately stabilize parameter binding via remapped seeds so that discrepancy across *different parameter names* is minimized; remaining variance is dominated by representation of the injected value itself. This scoping choice trades ecological completeness (full HTTP discrepancy surface) for interpretability of per-family SQLi mutation outcomes against one policy snapshot.

C. Defenses and Evaluation Gaps

Hybrid defenses aim to raise evasion cost through better data [8], shadow models and patch generation [5], or preprocessing [4]. HTTP-Normalizer exemplifies RFC-oriented validation as a discrepancy mitigation [3]. Such proposals, however, need *paired* offensive benchmarks: without logs that tie attempts to mutation families and outcome labels, it is difficult to tell

Seeds (PG/Oracle) → augmentations & mutate rules → optional 2nd-layer transforms → GET probes → JSONL + learned counters

Fig. 2. Offensive evaluation workflow implemented in the repository; dashed policy boundaries (rate limits, caps) are configurable in `config.json`.

whether a reported improvement removes high-yield operators or merely reshuffles search order. Our work supplies an open, log-grounded offensive baseline for such comparisons.

D. Black-Box Metrics and Threat of Confounding

Black-box WAF evaluation is inherently tied to *observable* outcomes: status codes, bodies, and timing. Different probes may map the same underlying vulnerability to different HTTP observables (e.g., generic 500 versus structured 422). Any operational definition of "bypass" therefore embeds epistemic commitments; transparent scripting and published logs mitigate but do not eliminate this issue. Section III states our probe's labels explicitly; Section V revisits threats to validity.

III. Methodology

This section specifies the adversarial evaluation methodology: adversary capabilities, staged mutation generation, HTTP probing, outcome labeling, and quantitative metrics. Artifacts reside under `paper/experiment`, including `ml-waf-aws-test` (live probing), shared generators under `ml-waf`, expansion from blocked traces (`ml-waf-blocked-adv`), offline surrogate scoring (`ml-waf-multi-eval`), and a compact RFC decoding pilot (`paper-defense-rfc-ml`).

A. Adversary Model

We model a network adversary who can send arbitrary HTTP/1.1 requests to a public origin fronted by a managed WAF, observe responses (status and body), and adapt future mutations based on logged outcomes. The adversary has no white-box access to WAF rules, ML model weights, or application source. This matches typical external penetration and red-team constraints and aligns with black-box evasion literature [1], [6].

B. Attack Pipeline

The end-to-end pipeline is:

$$\begin{aligned} \mathcal{M} : \text{Seeds} \to \text{Augmentations} \to \text{Transforms} \to \mathcal{V}, \\ \mathcal{V} \xrightarrow{\text{HTTP}} \text{WAF} \to \text{App} \to \text{Label}. \end{aligned} \tag{1}$$

Here $\mathcal{V}$ is a multiset of concrete request variants (the same logical payload may appear under different transform names or generations). The driver serializes each variant as a GET to the configured URL shape, records the response, and appends a JSON object to a line-oriented log.

C. Staged Mutation Generation

Generation is deliberately *staged* so that diversity and reproducibility can be traded off explicitly.

1) Seeds and Dialects: Oracle-oriented templates originate from the shared `variants_f5` corpus; PostgreSQL-oriented templates from `variants_pg`. Dialect choice affects function names, literal syntax, and comment styles, which in turn interact with WAF tokenization and application SQL parsers.

2) Deployment Augmentations and Learned Rules: The AWS harness can inject additional dialect- and encoding-specific variants and merge transforms described in `mutate_rules.json` (analogous to blocked-trace expansion in `ml-waf-blocked-adv`). Learned rules encode empirical regularities from prior blocked or bypass events; they bias the search toward regions of the space that previously produced informative outcomes.

3) Second-Layer Transforms and Search Bias: An optional second layer applies additional evasion transforms from a shared catalog, subject to caps and transform policy files. Consequently, the empirical distribution over families is *not* a uniform draw from all possible SQLi strings; it reflects queue scheduling, generation limits, and historical learning. Aggregate bypass rates must be read as properties of this *induced* distribution, not as an intrinsic property of "all SQLi." This is a feature for red teams (realistic search) and a caveat for comparability across papers (different search budgets yield different hit rates).

4) Remapping as an Experimental Control: Numeric seed tokens are remapped to stable string prefixes (e.g., project names) so that the WAF and application agree on the injected parameter slot. Without such alignment, high bypass counts could confound parameter shadowing, optional query keys ignored by the app, or divergent casing conventions—effects that are important in practice but orthogonal to the mutation operators under study.

D. Outcome Semantics

Let each attempt i produce HTTP status s_i and body b_i. The probe assigns $y_i \in \{\textsf{bypass}, \textsf{blocked}, \textsf{other}\}$ according to rules encoded in `run_probe.py` and companion documentation:

- **blocked**: explicit WAF or edge denial (e.g., 403 in the lab configuration).
- **bypass**: WAF-pass *and* application-visible signals classified as consistent with effective SQLi-side effects for the testbed (including selected error semantics and non-blocking status patterns defined in the script).
- **other**: ambiguous cases such as not-found or responses treated as ineffective noise (e.g., encoding-related 500s that do not meet the bypass predicate).

These labels are *operational*: they operationalize a red-team notion of "useful signal" rather than a formal proof of exploitability. They should not be equated with vendor marketing metrics or with ground-truth database compromise.

E. Metrics

Let N be the number of analyzed attempts and N_{bp} the count with $y_i = \textsf{bypass}$. The *aggregate bypass rate* is

$$\text{BR}_{\text{agg}} = \frac{N_{\text{bp}}}{N}. \tag{2}$$

For each mutation family f, let N_f be attempts tagged with f and $N_{f,\mathrm{bp}}$ bypasses among them. The *family-level bypass rate* is $\mathrm{BR}_f = N_{f,\mathrm{bp}}/N_f$. Export scripts also attach binomial confidence intervals to per-variant or per-family rates in machine-readable outputs; we report point estimates in the main tables and caution that families with tiny N_f produce unstable BR_f.

We additionally report *unique payload cardinalities*: distinct strings observed in bypass versus blocked sets. Overlap between sets indicates that the same surface form can yield different labels under context (e.g., timing, rate limiting, or state), motivating analysis at the attempt level rather than payload level alone.

F. Secondary Control: RFC Decoding and Surrogate Scoring

To isolate one normalization axis, we map each probe string through repeated `urllib.parse.unquote_plus` until a fixpoint (`unquote_chain`), then train/evaluate a random forest on a 20-row labeled JSON list. This tests whether a minimal RFC-relevant decoder aligns feature space enough for a toy surrogate to improve; it does not evaluate a full HTTP proxy [3], [16], [17].

G. Implementation Notes

The probe maintains per-transform counters (`learned_encodings.json`), supports dry-run and request caps, and can suggest regex fragments from logs for blue teams. Offline ModSecurity-style and RF scores (`ml-waf-multi-eval`) may disagree with the managed WAF; only live labels reflect the deployed policy.

IV. Experiments and Results

We answered RQ1–RQ3 using two datasets exported from our repository. The main dataset came from live AWS WAF probing, and the secondary dataset came from the RFC decoding surrogate pilot.

A. Experiment Setup and Environment

We ran all scripts on a Windows machine with version 10.0.26200 and 15.84 GB RAM. The experiment code is under `paper/experiment`, mainly `ml-waf-aws-test`, `ml-waf-blocked-adv`, `ml-waf-multi-eval`, and `ml-waf-adversarial-eval`. The Python dependencies used in these folders include `requests>=2.28.0`, `numpy>=1.24.0`, `scikit-learn>=1.3.0`, and `joblib>=1.3.0`. For the live run, we used `run_probe.py` and saved outputs to JSONL/TXT/JSON files under each module's `data` folder.

B. Attack Scenarios and What We Observed

We followed the same flow every time so the runs were comparable:

1) We started with SQLi seeds from PostgreSQL/Oracle style payload sets, then remapped IDs to the target parameter format.
2) We enabled mutation rules and optional second-layer transforms, then sent GET requests to the WAF-protected endpoint.
3) We logged each HTTP response as bypass, blocked, or other based on the classifier logic in `run_probe.py`.
4) We repeated this process at larger request counts and compared family-level behavior from exported summaries.

During probing, we observed two stable patterns: identity/case/null-byte families often passed at high rates, while encoding-heavy families were mostly blocked in this deployment.

C. Measurement Method

We measured three groups of outputs:

- **Attempt-level counts:** total requests, bypass count, and blocked count from `aws_probe_summary.json` and JSONL logs.
- **Family-level rates:** N_f and BR_f from exported tables in `paper-attack-evasion`.
- **Surrogate pilot accuracy:** test accuracy from `defense_eval.json` before and after `unquote_chain`.

Data collection was file-based. Every request appended one JSON line, then the reporting scripts aggregated those lines into summary JSON/CSV/TXT files. We kept all metrics exactly as exported and did not manually edit numeric outputs.

D. RQ1: Aggregate Scale, Labels, and Unique Payloads

The archived aggregate contains $N = 9{,}435$ HTTP attempts with $N_{\mathrm{bp}} = 3{,}721$ bypass-labeled outcomes, so $\mathrm{BR}_{\mathrm{agg}} = 0.394$ (39.4%). The mutation report for the same environment lists 3,724 bypass records and 5,687 blocked records. Unique bypass payloads are 42 and unique blocked payloads are 147, with 24 payloads appearing in both sets at least once.

This gap between attempts and unique payload strings means the search revisited similar strings under different transform labels or generations. The overlap between bypass and blocked sets also shows that payload text alone did not always decide the final outcome. Table I shows the corpus summary.

TABLE I
AWS WAF LAB ATTACK CORPUS (EXPORTED AGGREGATE).

Metric	Value	Notes
HTTP attempts N	9,435	Exported probe summary
Bypass count N_{bp}	3,721	$\text{BR}_{\text{agg}} = 39.4\%$
Unique bypass payloads	42	Mutation report
Unique blocked payloads	147	Mutation report
Payloads in both sets	24	Non-disjoint label sets

$\text{BR}_{\text{agg}} \approx 0.39$ over N=9,435; spawn_encoding cohort mostly blocked; identity/case/null-byte spawns high BR_f

Fig. 3. Archived live-probe summary: aggregate bypass rate and contrasting family-level behavior (Table II).

E. RQ1 (continued): Channel and Workload Shape

Our injection path used URL-visible slots with remapped SQLi fragments. The workload exercised percent-encoding changes, case changes, delimiter tricks, and dialect-specific spellings. This archived run does not directly test multipart `form-data`, JSON bodies, or XML bodies, so those channels need extra probes. Also, this dataset is attack-heavy by design, so it is not a benign baseline for false-positive analysis.

F. RQ2: Family-Level Heterogeneity

Figure 3 highlights the qualitative structure: aggregate bypass is substantial, but the mass is not spread evenly across operators.

Table II lists representative families with large N_f. The *spawn_identity*, *spawn_case_mix*, and *spawn_null_byte* cohorts stay around $\text{BR}_f = 0.89$–0.90. In contrast, *spawn_encoding* is around 0.025 and mostly blocked. In this deployment, that pattern matches a WAF policy that handled nested encodings more aggressively than case or null-byte suffix mutations. Hex/comment-oriented families often had $\text{BR}_f = 0$ at the attempted volumes in this export.

Statistical caution. Per-family rates are binomial proportions conditioned on the observed N_f for family f. Families with very small N_f (for example, two attempts) can show $\text{BR}_f = 1$ but still be unreliable. The exported analysis pipeline includes confidence intervals in CSV/JSON outputs, so those files should be checked when comparing families with very different attempt counts.

G. RQ3: Surrogate Accuracy Under RFC-Style Decoding

Table III summarizes the offline pilot. The random-forest test accuracy stayed at 42.9% for both raw strings and `unquote_chain`-canonicalized strings on the 20-request split in `defense_eval.json`. In this small sample, fixpoint percent-decoding did not improve or reduce surrogate performance.

H. Threats to Validity and Confounds

Label validity. Bypass labels encode a script-specific mapping from HTTP observables to outcomes; different applications would require recalibration.

Policy snapshot. AWS WAF rules, rate limits, and IP reputation evolve; our numbers describe one archived run.

Search bias. Queue order, caps, and learned transforms skew which families receive high N_f; BR_f compares families under that skewed design rather than under uniform random mutation.

Surrogate mismatch. The random forest is not a proxy for the managed WAF; RQ3 addresses only a narrow normalization hypothesis.

I. Measurement Overhead and Ethics

The harness prioritizes logging fidelity over maximum throughput, and sleep intervals plus request caps are configurable. All experiments were designed for *authorized* targets only; probing without authorization is illegal and unethical.

J. Reproducibility

Summaries can be regenerated with scripts under `ml-waf-aws-test/paper-attack-evasion` and `ml-waf-aws-test/paper-defense-rfc-ml`. Some JSON metadata may include absolute paths from the original workstation, so those paths should be updated after checkout. Running `run_probe.py` against a live endpoint will produce fresh logs, but counts will only match archived values if both configuration and WAF policy are the same.

V. Discussion and Conclusion

This work advances an attack-oriented, measurement-grounded view of SQLi WAF bypass. Rather than treating bypass as a scalar score, we decompose it along three axes emphasized throughout the paper: *semantics* (operational labels tied to observable HTTP/application signals), *scale* (attempt counts versus unique payload cardinality), and *structure* (family-level rates BR_f under an induced search distribution).

A. Answering the Research Questions

RQ1. The archived lab log yields $\text{BR}_{\text{agg}} = 39.4\%$ with $N = 9{,}435$, but only dozens of unique bypass strings, illustrating that high attempt-level rates can coexist with concentrated string reuse and partial overlap between bypass- and blocked-labeled payloads.

RQ2. Bypass probability is highly anisotropic across families: identity-, case-, and null-byte-oriented spawns achieve

TABLE II
SELECTED MUTATION FAMILIES (AWS WAF LAB EXPORT): ATTEMPTS N_f AND APPROXIMATE $\mathrm{BR}_f = N_{f,\mathrm{bp}}/N_f$.

Family	N_f	BR_f (%)	Role
spawn_identity	1,808	≈89.1	High yield
spawn_case_mix	459	≈89.8	High yield
spawn_null_byte	1,807	≈89.0	High yield
spawn_encoding	3,471	≈2.5	Mostly blocked
hex_account_style	2	≈100.0	Tiny N_f

TABLE III
OFFLINE SURROGATE PILOT (n=20 LABELED REQUESTS): TEST ACCURACY, RAW VS. RFC-STYLE CANONICALIZED STRINGS.

Condition	Test accuracy	Notes
Raw strings	0.429	Same seed and split
After `unquote_chain`	0.429	Fixpoint `unquote_plus`

$\mathrm{BR}_f \approx 0.89$–$0.90$ in the export, while an encoding-heavy spawn cohort is largely blocked ($\mathrm{BR}_f \approx 0.025$). For defenders, this implies that global block-rate dashboards can mask persistent holes in specific operator classes; regression suites should include family-targeted cases. For attackers (in authorized settings), the same table prioritizes mutation budget toward high-yield families for a given policy snapshot.

RQ3. Fixpoint percent-decoding does not improve a 20-example random-forest surrogate; live bypass therefore cannot be dismissed as an artifact of "the defender forgot to URL-decode once." Stronger mitigations must address richer feature interactions, policy coverage, and protocol-complete normalization where appropriate [3], [16].

B. Implications for Defenders

Policy tuning should be *family-aware*: after rule changes, re-run family-stratified probes rather than relying on a single aggregate metric. Combine WAF telemetry with application-layer controls (parameterized queries, least-privilege DB roles) because edge decisions will never be perfect. Where preprocessing is used, evaluate it against explicit attack logs rather than synthetic averages alone [4].

C. Implications for Offensive Research

Open logs enable third parties to dispute or refine labels, add families, or compare alternative outcome definitions—a step toward reproducible offensive security science. Publishing induced search configurations (caps, policies, seeds) is as important as publishing headline rates.

D. Ethics and Legal Scope

The methodology is intended solely for systems the operator owns or is authorized to test. Unauthorized scanning or bypass attempts violate law and policy; the artifact is a measurement instrument, not an endorsement of misuse.

E. Limitations and Future Work

Limitations include single-channel URL injection in the archived aggregate, single policy snapshot, and non-uniform search bias. Future work comprises multipart/JSON/XML probes, synchronized comparison of surrogate scores and live decisions on identical corpora, adaptive attackers that reweight families online, and pairing offensive logs with defensive rule synthesis tools already sketched in the repository (`generate_rules_from_data.py`).

F. Conclusion

We presented a reproducible black-box mutation and probing methodology for SQLi WAF evaluation, together with aggregate metrics, unique-payload structure, and family-resolved bypass rates from a live AWS WAF lab. The central methodological takeaway is that *depth* in offensive evaluation comes from explicit labels, decomposed statistics, and honest discussion of search bias and validity—not from a lone percentage. Family-level analysis reveals which mutation shapes dominate success under our conditions; a minimal RFC decoding control shows that trivial normalization does not resolve the offline surrogate pilot, reinforcing the need for layered defenses and richer benchmarks.

ACKNOWLEDGMENT

Write your acknowledgment here.

REFERENCES

REFERENCES

[1] Y. Guan et al., "SSQLi: A Black-Box Adversarial Attack Method for SQL Injection Based on Reinforcement Learning," *Future Internet*, vol. 15, no. 4, p. 133, 2023.

[2] S. A. Akhavani et al., "WAFFLED: Exploiting Parsing Discrepancies to Bypass Web Application Firewalls," in *Proc. Security Research Workshop*, 2025.

[3] S. A. Akhavani et al., "HTTP-Normalizer: A robust proxy tool for RFC-compliant HTTP validation," in *Proc. Security Research Workshop*, 2025.

[4] B. Zhang et al., "BWAFSQLi: Bypassing Web Application Firewall with Adversarial SQL Injections," *ACM Trans. Softw. Eng. Methodol.*, 2026.

[5] C. Wu et al., "WAFBOOSTER: Automatic Boosting of WAF Security Against Mutated Malicious Payloads," in *Proc. Int. Conf. Cybersecurity*, 2025.

[6] L. Demetrio et al., "WAF-A-MoLE: Evading Web Application Firewalls through Adversarial Machine Learning," in *Proc. 35th ACM/SIGAPP Symp. Appl. Comput. (SAC)*, 2020.

[7] L. M. Khan et al., “GSQLi: A GAN-based Approach for Adversarial SQL Injection Sample Generation against WAF,” in *Proc. Int. Conf. Inf. Secur.*, 2024.
[8] H. O. E. Elebiary et al., “Enhanced Web Payload Classification Using WAMM: An AI-Based Framework for Dataset Refinement and Model Evaluation,” *J. Inf. Secur. Appl.*, 2026.
[9] OWASP, “Core Rule Set (CRS) Documentation,” OWASP Foundation, 2020.
[10] M. Author, “A Survey of Web Application Firewall Technologies,” *J. Web Secur.*, 2019.
[11] T. Author, “Mutation-Based SQL Injection Evasion: An Empirical Study,” in *Proc. Web Secur. Conf.*, 2021.
[12] J. Author, “Parsing Discrepancies in Web Request Processing: Causes and Security Impact,” *IEEE Access*, 2022.
[13] K. Author, “Request Smuggling: Attacks and Defenses,” *Comput. Secur.*, 2020.
[14] R. Author, “Mutation Strategies for SQL Injection Payload Obfuscation,” in *Proc. Int. Symp. Secure Comput.*, 2021.
[15] S. Author, “Differential XML Parsing and Its Security Implications,” *J. Comput. Virol. Hacking Tech.*, 2022.
[16] R. Fielding et al., “RFC 7230: Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing,” IETF, 2014.
[17] T. Berners-Lee et al., “RFC 3986: Uniform Resource Identifier (URI): Generic Syntax,” IETF, 2005.
[18] M. Nottingham and R. Newton, “RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format,” IETF, 2017.
[19] S. Resnick and N. Khare, “RFC 7578: Returning Values from Multipart Form Data,” IETF, 2015.